AWS Lambda

What is AWS Lambda

1. AWS lambda is a serverless computing.
2. it is computing service which we can use to run our code without provisioning and managing the servers.
3. with aws lambda, you can run your code for virtually any type of application or backed service all with zero administration.
4. With AWS Lambda, you only pay for the compute time that you consume, which makes it a cost-effective solution for building scalable applications.
5. Additionally, AWS Lambda supports a variety of programming languages, making it easy for developers to write their code in the language they are most comfortable with.
6. AWS lambda runs our code on a high availability compute infrastructure.
7. AWS lambda executes our code when needed and scales automatically from a few requests per day to thousands per request.
8. we can pay only when our code run. No charges can apply when the code is not running.
9. all you need to do only supply your code to lambda in the form of one or more lambda function.
10. lambda supports the languages like node JS, python, go, ruby, c#, java, PowerShell, provided Runtime API (custom runtime).
11. AWS Lambda is often used to build serverless applications, which are applications that run in the cloud without requiring the developer to manage the underlying infrastructure.

CLOUDBLITZ

IMPORTANT POINTS

1. AWS Lambda runs a function for a maximum of 15 minutes (900 seconds) per execution.
2. If a Lambda function exceeds 15 minutes, AWS automatically terminates it.
3. Therefore, AWS Lambda is suitable only for short-running, event-driven tasks.
4. For long-running processes, services like EC2 or containers should be used instead.

KEY POINTS

1. Serverless computing
2. Run code on demand
3. Event driven
4. Automatic scaling
5. Cost effective
6. High availability
7. Support limited languages
8. Function based execution

How Lambda Works

1. first, you upload your code to lambda in one or more function.
2. then lambda executes the code
3. after the code is envoked, lambda automatically take care of provisioning and managing required servers.

Differences in AWS lambda and EC2

AWS Lambda

1. It is a platform as a service
2. It only supports the limited languages like java, python, ruby, PowerShell, c#, node JS, go, runtime API
3. In AWS lambda we can only for the time when the code runs

EC2

1. It is a infrastructure as a service
2. it has no environment so we can run the code on any languages
3. In EC2 we can pay per minute as well as pay per hour

Terms In Aws Lambda

Function:

1. a function is a resource that you can invoke to run your code in AWS lambda.
2. A function has code that processes events and a runtime that passes request and responses between lambda and the function code.

Runtime:

1. lambda runtimes allows functions in different languages to run in the same base execution environment.

CLOUDBLITZ

2. The runtime sits in between the lambda service and your function code relying invocation events, context information and responses between the two.

Event:

it is a JSON formatted text document that contains data for a function to process.

Event Source/ Trigger:

An AWS service such as Amazon SNS or a custom service that triggers your function and executes its logic.

Downstream Resource:

An AWS service, such as Dynamo DB tables or S3 buckets that your lambda function call once it is triggered.

Concurrency:

Number of requests that your function is serving in any given time.

Different way to invoke Lambda Function

1. Synchronous invoke (push)
2. Asynchronous invoke (event)
3. Poll-based invoke (pull based)

1. Synchronous Invoke:

1. Synchronous invoke are the most straight forward way to invoke your lambda function. In this model your functions execute immediately when you perform the lambda invoke API call.
2. Invocation flag specifies a value of “request-response”.
3. You wait for the function to process the event and return a response.

Here is a list of services that invoke lambda function synchronously:

1. Elastic Load Balancer
2. Amazon Cognito
3. Cloud front
4. API Gateway
5. Amazon Lex
6. Kinesis data firehose

2. Asynchronous Invoke:

1. For asynchronous invocation, lambda places the event in a queue and returns a success response without additional information.

2. Lambda queues the event for processing and returns a response immediately.
3. You can configure lambda to send an invocation record to another service like SQS, SNS, lambda and event bridge.

Here is a list of services that invoke lambda function asynchronously:

1. Amazon S3
2. Amazon SNS
3. SES
4. Cloud watch logs
5. Cloud watch events
6. AWS code commit
7. AWS config

3. Poll-Based Invoke:

The invocation model is designed to allow you to integrate with AWS stream and queue-based service with no code or server management lambda will poll the following service on your behalf, retrieve records and invoke your function.

The following are supported service:

1. Amazon Kinesis
2. Amazon SQS
3. Amazon Dynamo DB Streams

CLOUDBLITZ

S3 TRIGGER WITH LAMBDA

1.Create A Role For Lambda

1.type----------> aws service
2.use case------> lambda
3.policy----------> Administrator access
4.role name-----> role-for-lambda

2.Create A S3 Bucket

1.my-s3-bucket (bucket is private)

3.Go To Dynamodb

1.create a table

1. table-name---> newtable
2. partion key---> unique
3. create

4.Create A Lambda Function

1. author from scratch
2. name---------> lambda
3. runtime------> python3.10
4. advanced setting---role---existing role---(role-for-lambda)

5.Deploy Code On Lambda

1.paste the code on function

```
import boto3
from uuid import uuid4
dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('newtable')
def lambda_handler(event, context):
    for record in event['Records']:
        bucket_name = record['s3']['bucket']['name']
        object_key = record['s3']['object']['key']
        size = record['s3']['object'].get('size', 0)
        event_name = record['eventName']
        event_time = record['eventTime']
        table.put_item(
            Item={
                'unique': str(uuid4()),
                'Bucket': bucket_name,
                'Object': object_key,
                'Size': size,
                'Event': event_name,
                'EventTime': event_time
            }
        )
    return {
```

CLOUDBLITZ

```
    'statusCode': 200,
    'body': 'Success'
}
```

2.and deploy the code

5.Then Trigger the S3 Bucket

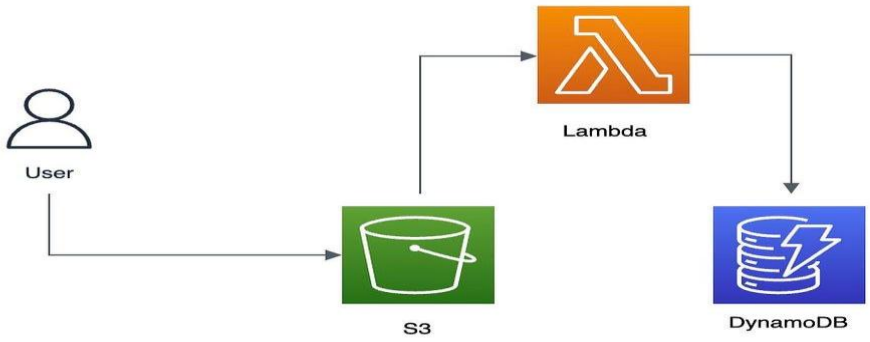
1. click on trigger
2. search---> s3
3. bucket---> my-s3-bucket
4. event----all-object-create-event
5. create a trigger

7.Add Objects In S3 Bucket

1. click on s3 bucket
2. add objects in the buckets

8.Go To DynamoDB

1. click on tables
2. click on newtable
3. click on explore item



CREATING SNAPSHOTS WITH PYTHON SCRIPT

1.create a instance in Mumbai region and in Singapore region

2.Create A Role

1. role----------aws service
2. use case-------lambda
3. policy----------Administrator Access
4. role-name----role-for-lambda

3.Create A Lambda Function In Lambda

1.author from scratch
2.function name-----> ebs-snapshot-function
3.runtime--------------> python3.10
4.role----> select existing role-----> role-for-lambda
5.create a function

4.On The Lambda Function

1.paste the code

```
import boto3

def lambda_handler(event, context):
   regions = ['ap-south-1', 'ap-southeast-1']  # Mumbai + Singapore

   for reg in regions:
      print(f"Checking region {reg}")

      ec2 = boto3.client('ec2', region_name=reg)

      volumes = ec2.describe_volumes(
         Filters=[{'Name': 'status', 'Values': ['in-use']}]
      )

      for volume in volumes['Volumes']:
         volume_id = volume['VolumeId']
         az = volume['AvailabilityZone']

         print(f"Creating snapshot for {volume_id} in {az}")

         snapshot = ec2.create_snapshot(
```

CLOUDBLITZ

```
            VolumeId=volume_id,
            Description='Lambda automated snapshot'
        )

        snapshot_id = snapshot['SnapshotId']

        # Tag snapshot
        ec2.create_tags(
            Resources=[snapshot_id],
            Tags=[
                {'Key': 'CreatedBy', 'Value': 'Lambda'},
                {'Key': 'VolumeId', 'Value': volume_id}
            ]
        )

    return {
        'statusCode': 200,
        'body': 'Snapshots created successfully'
    }
```

2.Deploying the code

3.Go To Configuration And Edit

1.memory----- 512MB

2. Timeout ---- 3–5 minutes

3.save

4.Test Lambda

1. create a new test event

```
{
 "key": "test"
}
```

2. test again

5.Verify Snapshots

1. Go to Mumbai region & Singapore region
2. EC2 → Snapshots
3. You should see snapshot created

EVENTBRIDGE TRIGGER WITH LAMBDA

1. CREATE IAM ROLE FOR LAMBDA

1. role ----> role-for-lambda-eventbridge

2.CREATE LAMBDA FUNCTION

1. Create a Function
2. Author from Scratch
3. Function Name → ec2-state-monitor
4. Runtime → Python 3.10
5. Existing Role (role-for-lambda-eventbridge)
6. Create Function

3. DEPLOY PYTHON CODE

```
import json

def lambda_handler(event, context):

    instance_id = event['detail']['instance-id']
    state = event['detail']['state']

    print(f"EC2 Instance {instance_id} changed state to {state}")

    return {
        'statusCode': 200,
        'body': json.dumps('EC2 state captured successfully')
    }
```

CLOUDBLITZ

4. ADD EVENTBRIDGE TRIGGER

1. click on trigger
2. Trigger configuration --------> EventBridge (CloudWatch Events)
3. rule ------> Create a new rule
4. rule_name ----> ec2-state-rule
5. Rule type ------> Event pattern
    1. EC2
    2. All Events
6. add trigger

5. CREATE EC2 INSTANCE

6. TEST THE PRACTICAL

1. Stop Instance
2. Start Instance
3. Reboot Instance

7. VERIFY CLOUDWATCH LOGS

1. Click on Lambda → ec2-state-monitor
2. Click on Monitor Tab
3. Click on View CloudWatch Logs
4. Open Latest Log Stream

8. OUTPUT YOU RECEIVED

1. EC2 Instance i-088522fbea2f6ab23 changed state to pending
2. EC2 Instance i-088522fbea2f6ab23 changed state to running
3. EC2 Instance i-088522fbea2f6ab23 changed state to stopping

---

CLOUDBLITZ